

Project Module 3

Robustness Testing Safe Deployment

German Credit Dataset

Adham Ali	900223243
Omar Saqr	900223343
Ebram Thabet	900214496
Saif Abd Elfattah	900225535

Submitted to Dr. Alia El Bolock
CSCE 4930 – Department of Computer Science and Engineering
The American University in Cairo

April 29, 2026

Contents

1	Part 1: Baseline Model and Clean Evaluation (10%)	2
1.1	Data and Preprocessing	2
1.2	Baseline Model	2
1.3	Clean Test Performance	2
1.4	Analysis	4
2	Part 2: Robustness Stress Testing (30%)	5
2.1	Perturbation Protocol	5
2.2	Results	5
2.3	Analysis	7
3	Part 3: Fairness and Explanation Stability (20%)	9
3.1	Method	9
3.2	Prediction Flip Rates by Subgroup	9
3.3	Demographic Parity Under Perturbation	10
3.4	Equalized Odds Under Perturbation	11
3.5	Feature Importance Stability	12
3.6	Calibration	12
3.7	Analysis	14
4	Part 4: Defense Implementation (15%)	17
4.1	Defense Choice	17
4.2	Implementation	17
4.3	Results	17
4.4	Analysis	18
5	Part 5: Analysis and Recommendation (25%)	19

Part 1: Baseline Model and Clean Evaluation (10%)

Data and Preprocessing

We use the German Credit Dataset (1,000 samples). The preprocessing pipeline is the same one from Modules 1 and 2.

- **Target:** `credit_risk - 1` = Good credit, 0 = Bad credit.
- **Split:** 80% train (800 samples) / 20% test (200 samples), stratified on the target, `random_state = 42`.
- **Numeric features:** fill missing values with the median, then scale with `StandardScaler`.
- **Categorical features:** fill missing values with the most frequent value, then one-hot encode.
- After preprocessing, the dataset has 65 features.
- The columns `sex`, `age_group`, and `foreign_worker` are kept in the model, same as Module 2. A separate slice (`A_test`) stores them for subgroup checks in later parts.

Baseline Model

We train a Decision Tree classifier with the same settings used in Module 2:

```
max_depth = 5, criterion = gini, min_samples_leaf = 10, random_state = 42
```

The tree is trained only on the training split. No test data is used during training. Feature importances are stored in a `TREE_IMPORTANCE` dictionary for use in Part 2. The most important feature is `status_no_checking_account` with a Gini importance of 0.3662 (36.6% of total tree importance), which is also the root split.

Clean Test Performance

Table 1 shows the model's performance on the clean test set. Figure 1 shows the same metrics as a bar chart. Figure 2 shows the confusion matrix.

Table 1: Baseline performance on the clean test set (200 samples).

Metric	Class	Value
Accuracy	Both classes	0.7100
Precision	Good (1)	0.7253
Recall	Good (1)	0.9429
F1-score	Good (1)	0.8199
Precision	Bad (0)	0.5600
Recall	Bad (0)	0.1700
F1-score	Bad (0)	0.2600
F1 (macro)	Both classes	0.5381

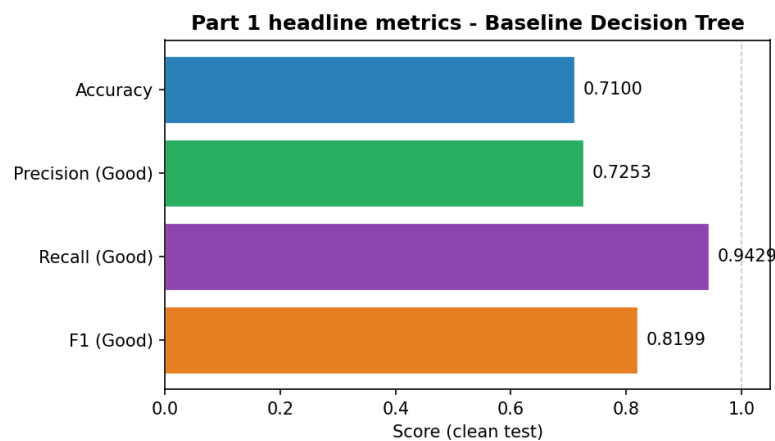


Figure 1: Baseline metrics bar chart. The Good class recall is 0.94, which is high. The Bad class recall is only 0.17, meaning the model misses most risky applicants. This gap is the main weakness of the baseline.

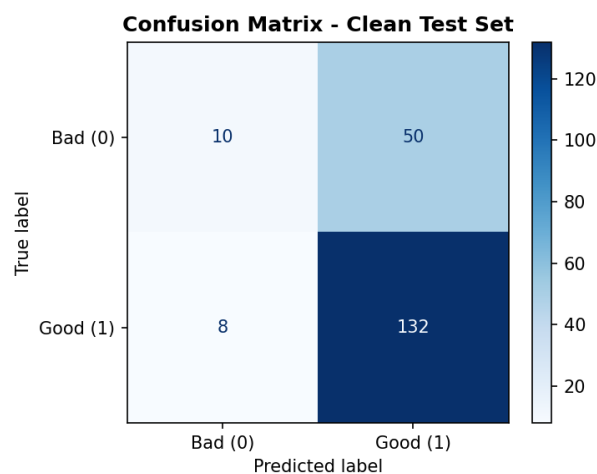


Figure 2: Confusion matrix on the clean test set. Out of 60 truly Bad applicants, only 10 are correctly identified. Out of 140 truly Good applicants, 132 are correctly identified. The model heavily favours approving applicants over rejecting them.

Analysis

Question 1: What is the baseline performance of the model?

The model reaches an accuracy of 0.71 on the clean test set. The full breakdown is in Table 1.

For the Good class, recall is 0.94, meaning the model correctly approves 94% of truly creditworthy applicants. Precision for the Good class is 0.73, meaning some approved applicants are actually bad risks.

For the Bad class, recall is only 0.17. The model misses 83% of truly risky applicants and approves them by mistake. In a real credit system, this could lead to large financial losses. The problem comes from the 70/30 class distribution: the model learns to predict Good most of the time because Good applicants are more common. This matches the pattern found in Module 2.

The macro F1 of 0.54 shows that when both classes are weighted equally, the model performs much worse than the headline accuracy of 0.71 suggests.

Question 2: Why is performance on clean data not enough to trust a model?

A good score on clean test data only tells us the model works on ideal, controlled data. It does not mean the model is safe or fair in the real world.

1. **Real data is messy.** Real loan applications have noise, missing fields, typos, and people who try to game the system. Clean test data has none of that. Part 2 of this report tests some of these cases.
2. **The world changes.** The German Credit Dataset is from the 1990s. A model trained on it may not work well on applicants today, who have different financial habits and economic conditions.
3. **Overall metrics hide group-level harm.** An accuracy of 0.71 can hide the fact that the model treats some groups much worse, for example young applicants or foreign workers. This matters for fairness and regulation.
4. **Trust needs more than one number.** Banks and regulators need explanations, monitoring over time, and ways for applicants to appeal decisions, not just a single offline metric.

Clean metrics are a starting point, not a final answer. They give us a reference for the stress tests that follow.

Part 2: Robustness Stress Testing (30%)

Perturbation Protocol

We apply four different changes to the test data and evaluate the same model from Part 1 on each changed version. The training data is never changed. The model is never retrained. All random steps use `PERTURB_SEED = 12345` for reproducibility.

Scenario A – Gaussian noise. We add random noise (standard deviation = $0.22 \times$ training σ) to all numeric features. This simulates small data entry errors and measurement noise in real applications.

Scenario B – Coarsening. We round `amount` and `duration` to the nearest 100, and all other numeric features to the nearest whole number. This simulates reduced precision in data collection. Because `duration` values in this dataset range from 4 to 72 months, rounding to the nearest 100 maps every duration to 0, effectively removing the feature.

Scenario C – Missing values + re-imputation. We randomly set 15% of numeric feature values to missing, then fill them with the column median. This simulates incomplete loan applications where applicants leave fields blank.

Scenario D – Feature manipulation. We swap the `status` category values “no checking account” and “... < 100 DM” for each other. This targets the most important feature in the tree (the root split, 36.6% of Gini importance). It simulates an applicant who deliberately misreports their checking account status to improve their approval chances.

For each scenario, we report accuracy, error rate ($= 1 - \text{accuracy}$), their delta from baseline, F1 for the Good class, F1 macro, and the flip rate (fraction of predictions that changed compared to the clean run).

Results

Table 2 shows the exact results from the notebook run. Figures 3, 4, and 5 show the results visually.

Table 2: Model performance under each perturbation. All values are exact outputs from the notebook run.

Scenario	Accuracy	Δ Acc	Error Rate	F1 (Good)	F1 (macro)	Flip Rate
Baseline (clean)	0.7100	0.0000	0.2900	0.8199	0.5381	0.000
A) Gaussian noise	0.7100	0.0000	0.2900	0.8210	0.5289	0.040
B) Coarsening	0.6800	-0.0300	0.3200	0.7935	0.5412	0.150
C) Missing + re-impute	0.7150	+0.0050	0.2850	0.8257	0.5224	0.025
D) Status swap	0.7050	-0.0050	0.2950	0.8127	0.5593	0.095

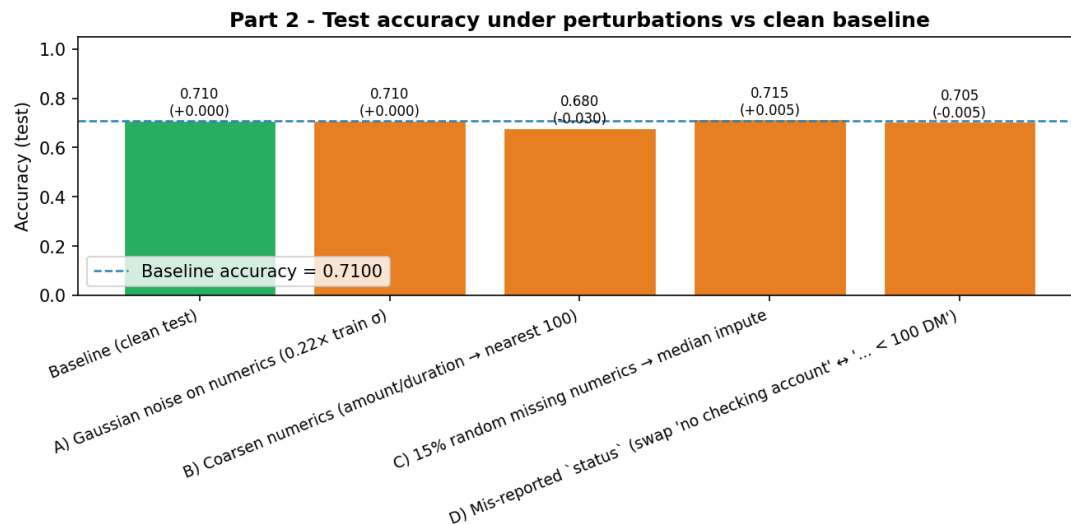


Figure 3: Accuracy under each scenario compared to the clean baseline (dashed line). Scenario B (coarsening) causes the largest accuracy drop, falling from 0.71 to 0.68. Scenarios A and C stay at or above baseline. Scenario D shows only a small drop of 0.005.

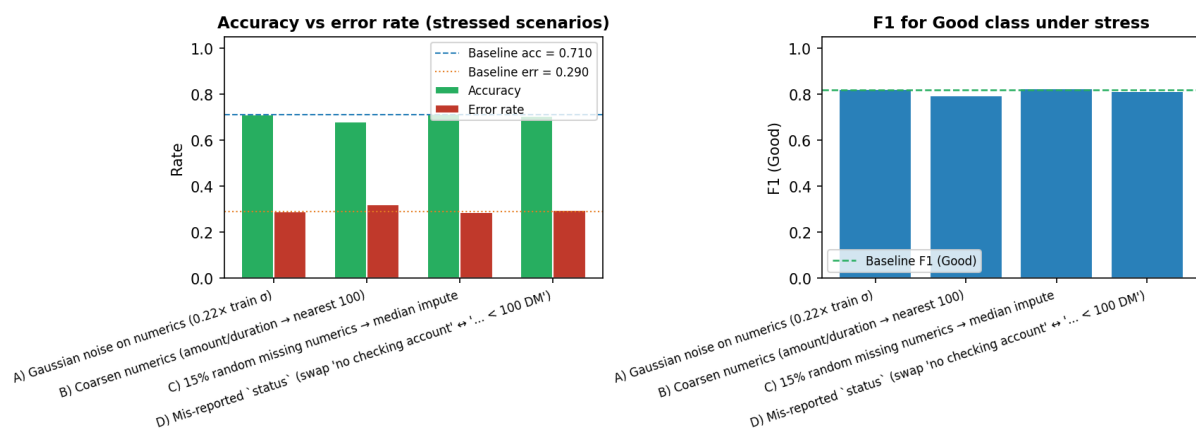


Figure 4: Left panel: accuracy (green) and error rate (red) for the four stressed scenarios, with baseline reference lines. Right panel: F1 for the Good class under each scenario. Scenario B shows the clearest drop in both accuracy and F1. Scenario C stays above baseline on both metrics.

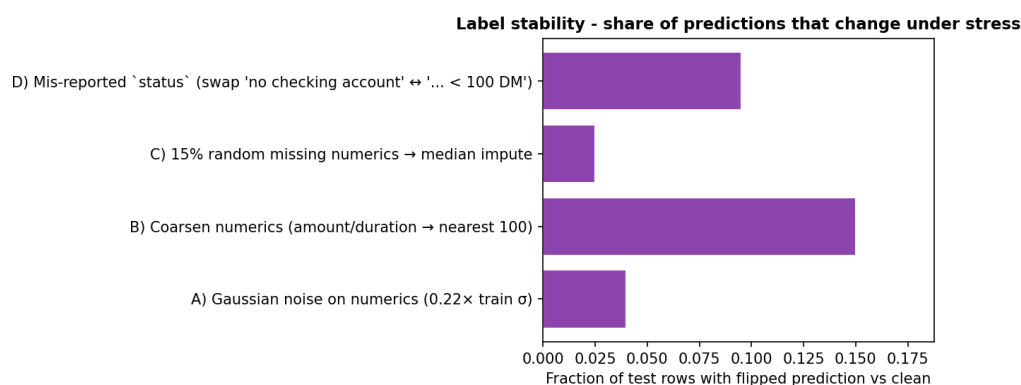


Figure 5: Prediction flip rate per scenario. Scenario B has the highest flip rate at 15%, meaning 30 out of 200 predictions changed. Scenario C has the lowest at 2.5%, confirming that median re-imputation causes very little instability.

Analysis

Question 1: Which perturbation affected performance the most?

Scenario B (coarsening) caused the largest accuracy drop: from 0.71 to 0.68, a change of -0.03 . It also had the highest flip rate at 15%, meaning 30 out of 200 predictions changed.

The reason is how coarsening affects this specific dataset. The `duration` feature has values between 4 and 72 months. Rounding to the nearest 100 maps every single duration value to 0. This completely wipes out the information in one of the top-3 most important features: `duration` has a Gini importance of 0.1585, second only to `status`. Any split in the tree that uses `duration` now receives 0 for all applicants, pushing many of them to the same branch regardless of their actual loan length. For `amount`, rounding to the nearest 100 is less extreme but still shifts some values across thresholds.

Scenario D (status swap) had a smaller effect than expected: only -0.005 accuracy drop and a 9.5% flip rate. Although 120 out of 200 rows had their status swapped, many applicants were still correctly classified through other splits further down the tree. This shows the model does not depend solely on the root split for every prediction.

Scenario C (missing values) slightly improved accuracy by $+0.005$. Median re-imputation happened to move some borderline cases to the correct side of a decision boundary.

Question 2: Did small changes cause large prediction differences?

Yes, in some cases. Scenario A (Gaussian noise with $\sigma = 0.22 \times \text{training } \sigma$) is a small perturbation, yet it still flipped 4% of predictions (8 out of 200) without changing overall accuracy. This means some predictions changed from wrong to right and others changed from right to wrong in equal measure, leaving accuracy unchanged while individual pre-

dictions moved.

The clearest example is Scenario B. Rounding numbers to coarser values looks minor, but because it eliminates the duration feature entirely, it affects 15% of predictions and drops accuracy by 3 points.

A Decision Tree uses hard yes/no splits at each node. A feature value very close to a threshold only needs a tiny change to cross it. Once it crosses, the applicant goes down a completely different branch, which often leads to a different prediction. This is called boundary sensitivity. It explains why small input changes can flip individual predictions even when overall accuracy stays similar.

Question 3: What does this suggest about model reliability?

The clean accuracy of 0.71 only reflects performance on ideal data. Under stress, it drops to 0.68 and up to 15% of predictions change. This reveals several reliability concerns.

The model is sensitive to precision loss in numeric features. Scenario B shows that reducing the precision of just two features (`amount` and `duration`) drops accuracy by 3 points. In a real deployment, data collected from different systems or time periods may have varying levels of numeric precision. The model gives no warning when this happens – it just produces different predictions silently.

The model also has boundary sensitivity from its hard threshold structure. A Decision Tree cannot express uncertainty near decision boundaries. It always gives a definitive approve or reject, even for applicants whose features sit right on the edge. This means two nearly identical applicants can receive opposite decisions based on data imprecision rather than genuine differences in creditworthiness.

One positive finding is that the model is more resilient to the status swap (Scenario D) than expected. Despite 120 rows being affected, accuracy dropped by only 0.005. This suggests that other features along the decision path help recover correct answers for many applicants, and the model does not collapse when its most important feature is corrupted.

To deploy this model safely, the following steps would be needed: standardize and validate numeric inputs before scoring, especially `duration` and `amount`; verify high-stakes categorical fields such as `status` against bank records rather than relying on applicant self-report; monitor model performance continuously after launch; require human review on borderline decisions; and consider retraining with an ensemble method or stronger regularization to reduce sensitivity to numeric precision loss. A clean test accuracy of 0.71 is not a sufficient basis for full automation.

Part 3: Fairness and Explanation Stability (20%)

This part checks whether the robustness problems found in Part 2 affect all applicants equally, or whether certain demographic groups are hit harder than others. It also checks whether the model’s explanations (feature importances) remain consistent when the input data is perturbed. The three sensitive attributes from Module 2 are used: **sex** (male / female), **age_group** (25 and over / under 25), and **foreign_worker** (yes / no). These groups are stored in the **A_test** slice from Part 1.

Method

For each of the four perturbation scenarios from Part 2 (Gaussian noise, coarsening, missing values + re-imputation, and status swap), we break down two quantities by subgroup:

1. **Flip rate:** the fraction of predictions in each subgroup that changed compared to the clean baseline. A higher flip rate means the subgroup’s outcomes are less stable.
2. **Selection rate:** the fraction of applicants in each subgroup who receive a “Good” (approve) prediction. The gap between the highest and lowest selection rates across groups is the demographic parity gap. A larger gap means the model treats the groups more unequally.

Feature importance is taken directly from the trained Decision Tree (Gini importance), which was computed once during training in Part 1 and does not change across perturbation scenarios because the model is never retrained.

Prediction Flip Rates by Subgroup

Table 3 shows how often predictions changed (flipped) in each subgroup under each perturbation. Figure 6 shows the same data visually.

Table 3: Prediction flip rates by subgroup under each perturbation. Higher values mean less stable predictions for that group.

Scenario	Female	Male	25+	Under 25	Foreign: No	Foreign: Yes
A) Gaussian noise	0.050	0.036	0.029	0.107	0.000	0.042
B) Coarsening	0.367	0.057	0.140	0.214	0.000	0.157
C) Missing + re-impute	0.033	0.021	0.023	0.036	0.000	0.026
D) Status swap	0.100	0.093	0.087	0.143	0.222	0.089

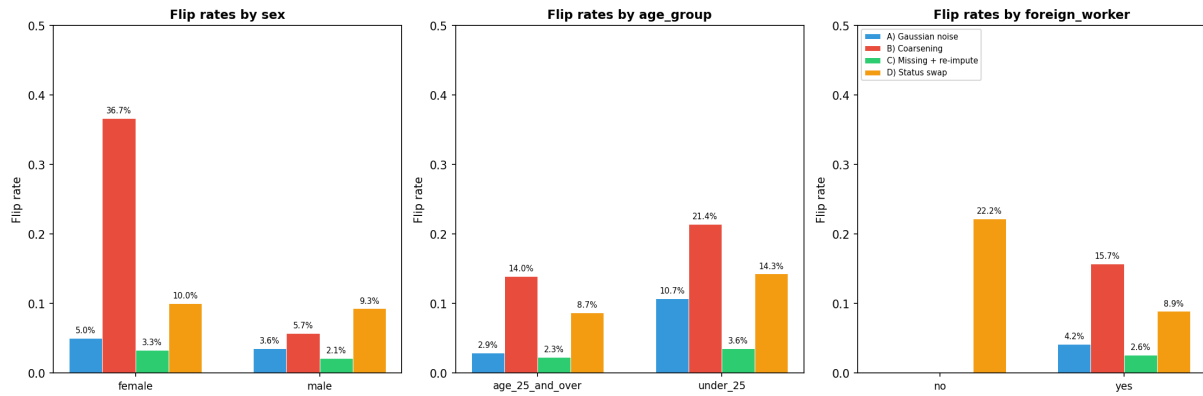


Figure 6: Prediction flip rates by subgroup for each perturbation scenario. Scenario B (coarsening) produces the largest disparity: 36.7% of female predictions flip, compared to only 5.7% of male predictions. Under-25 applicants also show consistently higher flip rates than the 25+ group across all four scenarios.

Demographic Parity Under Perturbation

Table 4 shows the selection rate for each subgroup and the demographic parity (DP) gap under each scenario. The DP gap is the difference between the highest and lowest selection rates within an attribute. Figure 7 shows the gaps visually.

Table 4: Selection rates and demographic parity gaps by sensitive attribute under each scenario. A larger gap means the model treats the two groups more differently.

Scenario	Sex			Age group			Foreign worker		
	F	M	Gap	25+	<25	Gap	No	Yes	Gap
Baseline (clean)	0.850	0.936	0.086	0.919	0.857	0.062	1.000	0.906	0.094
A) Gaussian noise	0.833	0.957	0.124	0.936	0.821	0.115	1.000	0.916	0.084
B) Coarsening	0.517	0.993	0.476	0.872	0.714	0.158	1.000	0.843	0.157
C) Missing + re-impute	0.883	0.957	0.074	0.942	0.893	0.049	1.000	0.932	0.068
D) Status swap	0.850	0.886	0.036	0.878	0.857	0.021	0.778	0.880	0.102

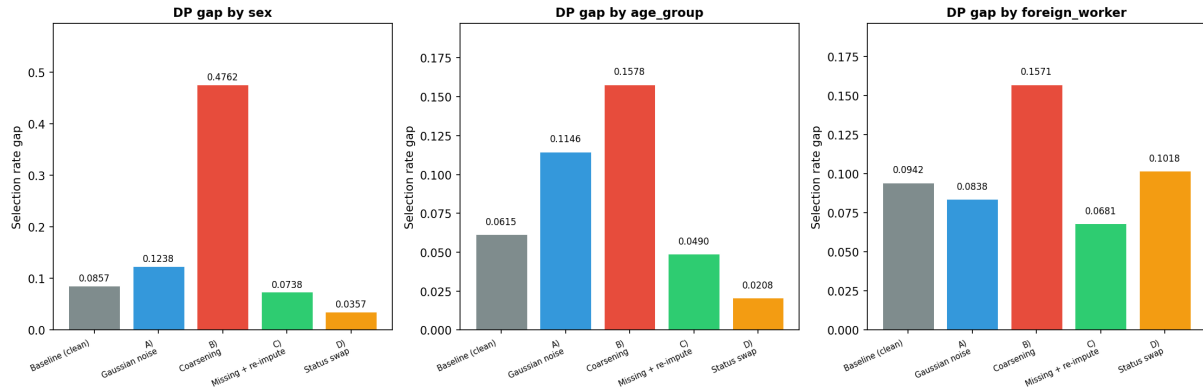


Figure 7: Demographic parity gap for each sensitive attribute under each scenario. Scenario B (coarsening) causes the sex-based gap to jump from 0.086 on clean data to 0.476 – a $5.5\times$ increase. This means coarsening makes the model far more discriminatory by sex than any other perturbation.

Equalized Odds Under Perturbation

Equalized odds requires that both the true positive rate (TPR) and the false positive rate (FPR) are equal across groups. The equalized odds gap is the larger of the two differences. Table 5 shows TPR, FPR, and the equalized odds gap by sex under each scenario.

Table 5: Equalized odds by sex. TPR = fraction of truly Good applicants correctly approved. FPR = fraction of truly Bad applicants incorrectly approved. EO gap = $\max(\text{TPR gap}, \text{FPR gap})$.

Scenario	Female		Male		TPR gap	EO gap
	TPR	FPR	TPR	FPR		
Baseline (clean)	0.900	0.750	0.960	0.875	0.060	0.125
A) Gaussian noise	0.875	0.750	0.980	0.900	0.105	0.150
B) Coarsening	0.575	0.400	1.000	0.975	0.425	0.575
C) Missing + re-impute	0.950	0.750	0.970	0.925	0.020	0.175
D) Status swap	0.925	0.700	0.910	0.825	0.015	0.125

On clean data the equalized odds gap by sex is 0.125, driven by the FPR difference (0.875 for males versus 0.750 for females). Under Scenario B (coarsening), the gap jumps to 0.575. The driving factor is the FPR: male FPR rises to 0.975 (nearly every Bad male applicant is wrongly approved), while female FPR drops to 0.400 (female Bad applicants are now more likely to be correctly rejected). At the same time, female TPR falls from 0.900 to 0.575 – meaning 42.5% of genuinely creditworthy female applicants are now wrongly rejected. Males lose nothing: their TPR rises to 1.000.

In practical terms, coarsening makes the model simultaneously reject good female applicants and approve bad male applicants, widening the disparity on both axes of equalized

odds. This is the kind of hidden bias that would not appear in aggregate accuracy (which only drops by 3 percentage points overall) but shows up clearly in subgroup-level TPR and FPR.

Feature Importance Stability

The feature importances used in this analysis come from the Gini importance scores of the trained Decision Tree. Because the model is trained only once (on the clean training set) and is never retrained under any perturbation scenario, the importance scores themselves do not change. The top three features remain **status_no_checking_account** (0.3662), **duration** (0.1585), and **amount** (0.1103) regardless of what happens to the test data.

However, the fact that the importance values stay constant does not mean the model’s behaviour is stable. Feature importance tells us how the tree was built, not how the tree performs on new data. When Scenario B rounds **duration** to the nearest 100, every duration value maps to 0. The tree still says duration is the second most important feature, but that information is now useless because every applicant gets the same duration value at scoring time. The tree’s splits on duration become meaningless, and the predictions change for 15% of applicants overall – and for 36.7% of female applicants specifically.

This disconnect between static importance and actual prediction behaviour is a key finding. A model can report the same explanation (“duration is important”) while producing very different outcomes for different groups. In other words, feature importance alone is not a reliable fairness or stability check.

Calibration

Calibration measures whether the model’s predicted probabilities reflect actual outcomes. If the model outputs a probability of 0.8 for a Good credit decision, roughly 80% of applicants in that bin should actually be Good. Poor calibration means the model’s confidence is misleading, which matters when scores feed into downstream systems or are shown to decision-makers. We use Platt scaling (**CalibratedClassifierCV**) on the already-trained Decision Tree – the tree structure and weights do not change; only a thin logistic layer is fitted to map raw scores to probabilities.

The overall Brier score on the test set is 0.1968, reflecting moderate calibration quality consistent with a shallow tree. Figure 8 shows the calibration curve for the full test set.

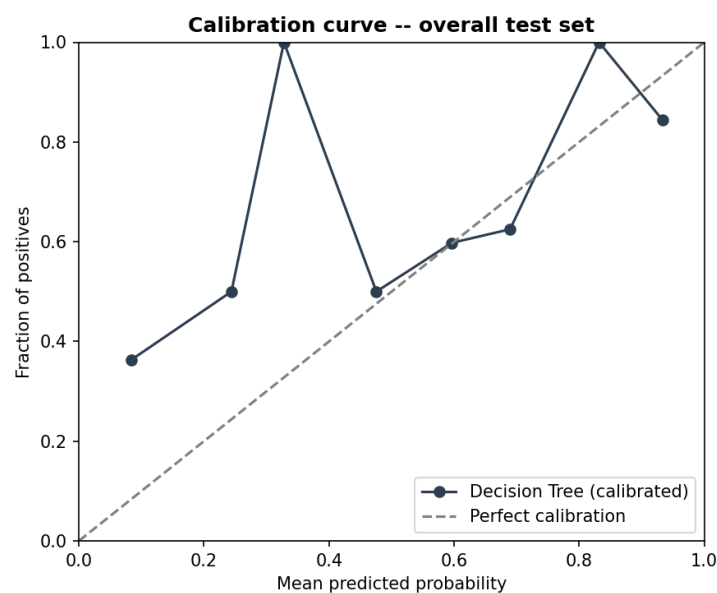


Figure 8: Overall calibration curve. Points above the diagonal mean the model underestimates risk; points below mean it overestimates. The model is reasonably calibrated at low probabilities but overestimates confidence in the highest-probability bin.

Figure 9 shows calibration broken down by sex.

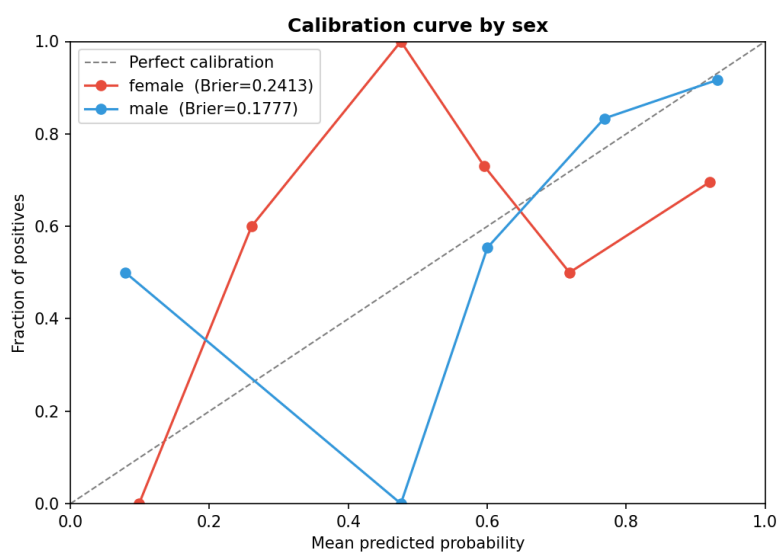


Figure 9: Calibration curves by sex. Female Brier score (0.2413) is higher than male (0.1777), meaning the model's probability estimates are less accurate for female applicants even on clean data. The female curve diverges further from the diagonal in the mid-range bins, where most borderline lending decisions fall.

The female Brier score of 0.2413 versus 0.1777 for males is a 35.8% relative difference in calibration error. Even before any perturbation, the model is less trustworthy for female applicants. If a bank uses the model's probability output to set interest rate tiers or flag

borderline cases for human review, this calibration gap would systematically disadvantage female applicants – they would be placed in the wrong risk category more often.

Analysis

Question 1: Did robustness issues disproportionately affect any group?

Yes. The most striking example is the effect of Scenario B (coarsening) on female applicants. Out of the 60 female applicants in the test set, 22 (36.7%) had their predictions flipped, compared to only 8 out of 140 male applicants (5.7%). This is a ratio of roughly 6 to 1. Female accuracy dropped from 0.68 to 0.58, while male accuracy stayed at 0.72.

The reason is tied to the features that coarsening destroys. Duration and amount are both in the top three most important features in the tree. Rounding duration to the nearest 100 maps every value to 0, which removes the feature entirely. Female applicants in this dataset are more concentrated around decision boundaries that depend on these numeric features, so when those features lose their precision, a much larger share of female predictions cross a split threshold and end up on the wrong branch.

Under-25 applicants are also consistently more affected than the 25+ group. In Scenario B, the under-25 flip rate is 21.4% versus 14.0% for the 25+ group. In Scenario A (Gaussian noise), the gap is even wider: 10.7% versus 2.9%. This pattern makes sense because under-25 applicants already have lower baseline accuracy (0.50 versus 0.74 for the 25+ group), which means they are already closer to the decision boundaries and therefore more vulnerable to perturbation.

For the foreign worker attribute, non-foreign workers (only 9 in the test set) show zero flips for Scenarios A, B, and C, but a 22.2% flip rate for Scenario D (status swap). This is because 2 of the 9 non-foreign workers had their status swapped, and the small group size makes the percentage appear large. The foreign worker group (191 applicants) has more moderate flip rates that follow the overall trend.

Question 2: Did explanations remain consistent under perturbations?

The model's reported feature importances did not change, because the tree was trained once and never retrained. However, these static importances are misleading. They describe how the tree was built, not how it actually behaves on a given test set.

Consider Scenario B: the tree still reports **duration** as the second most important feature (Gini importance 0.1585), but every test applicant now has a duration of 0 after coarsening. The splits on duration no longer separate applicants in a meaningful way – everyone goes down the same branch. The “explanation” (duration matters) is technically still reported, but it no longer reflects reality.

This means that while the feature importance numbers are stable, the explanations they provide are not reliable under perturbation. A user looking at the model's explanation would still be told that duration is important, even though the feature has been rendered useless by data imprecision. This is a form of explanation instability: the model gives the same explanation regardless of whether it actually holds for the data at hand.

A more reliable stability check is to look at prediction flip rates per subgroup (Table 3). If a perturbation changes 36.7% of one group's predictions but only 5.7% of another's, the model's behaviour is clearly not consistent across groups, even if the reported feature importances have not moved.

Question 3: Why does instability create ethical concerns?

Instability creates ethical concerns because it means the model's decisions depend on factors outside of any individual applicant's control, and these dependencies are not equally distributed across demographic groups.

1. **Unequal exposure to harm.** Scenario B shows that a simple reduction in numeric precision – something that could happen due to a data entry system change, a format conversion, or a third-party data feed update – would flip 36.7% of female predictions but only 5.7% of male predictions. Female applicants would disproportionately lose access to credit through no fault of their own. This violates the fairness principle that similar applicants should receive similar outcomes.
2. **Hidden discrimination.** The demographic parity gap by sex goes from 0.086 on clean data to 0.476 under coarsening. On clean data, the model approves 85% of female applicants and 93.6% of male applicants – a gap that might be considered acceptable. Under coarsening, it approves only 51.7% of female applicants while approving 99.3% of males. This level of disparity would likely violate anti-discrimination standards, and it happens silently. The model does not flag that its fairness properties have changed. Without ongoing monitoring, no one would notice until the damage is done.
3. **Erosion of trust.** When two nearly identical female applicants can receive opposite credit decisions because one's loan duration was recorded as 24 months and the other's was rounded to 0, the system cannot be considered trustworthy. Applicants have no way to know whether their outcome was driven by genuine creditworthiness or by a rounding artefact. This undermines the legitimacy of the decision-making process.
4. **No warning mechanism.** The model does not know when its inputs have been degraded. It produces a confident approve or reject decision regardless of whether the input data is clean or corrupted. Without external monitoring that tracks both overall and per-group performance over time, instability-driven harm can go undetected. This

is especially dangerous because the groups most affected (female applicants, under-25 applicants) are also the groups with less statistical representation in the dataset, making their degradation harder to spot in aggregate metrics.

To address these concerns, deployment would need to include at minimum: ongoing fairness monitoring that tracks selection rates and flip rates by subgroup; input validation to catch precision loss before scoring; alerting when demographic parity gaps exceed a defined threshold; and human review for applicants near decision boundaries.

Part 4: Defense Implementation (15%)

Defense Choice

We use one simple defense: **training with augmented data**. Instead of changing the model family, we keep the same Decision Tree and the same preprocessing pipeline from Parts 1–3. The only change is that we retrain the tree on a larger training set that includes:

- the original clean training data,
- a copy with Gaussian noise added to numeric features,
- a copy with rounded numeric values,
- a copy with random missing numeric values that are later imputed,
- a copy with the **status** feature swapped in the same way as Scenario D.

This increases the training set from 800 rows to 4,000 rows. The idea is simple: if the model sees these small changes during training, it should become less fragile when similar changes appear at test time.

Implementation

The defended model still uses:

```
max_depth = 5,  criterion = gini,  min_samples_leaf = 10,  random_state = 42
```

So the dataset, preprocessing steps, and model family all stay consistent with Modules 1 and 2, as required. Only the training data is augmented before fitting.

Results

Table 6 compares the baseline tree from Part 1 with the defended tree on the clean test set and on the four perturbation scenarios from Part 2. Figure 10 shows the accuracy comparison visually.

Table 6: Before/after comparison for the defense in Part 4. All values are exact outputs from the notebook run.

Scenario	Base Acc	Def Acc	Δ Acc	Base F1 macro	Def F1 macro	Δ F1 macro	Base Flip	Def Flip
Clean	0.7100	0.7500	+0.0400	0.5381	0.6573	+0.1192	0.000	0.000
A) Gaussian noise	0.7100	0.7200	+0.0100	0.5289	0.6162	+0.0873	0.040	0.040
B) Coarsening	0.6800	0.7200	+0.0400	0.5412	0.5921	+0.0509	0.150	0.070
C) Missing + re-impute	0.7150	0.7350	+0.0200	0.5224	0.6230	+0.1006	0.025	0.035
D) Status swap	0.7050	0.7150	+0.0100	0.5593	0.6121	+0.0528	0.095	0.155

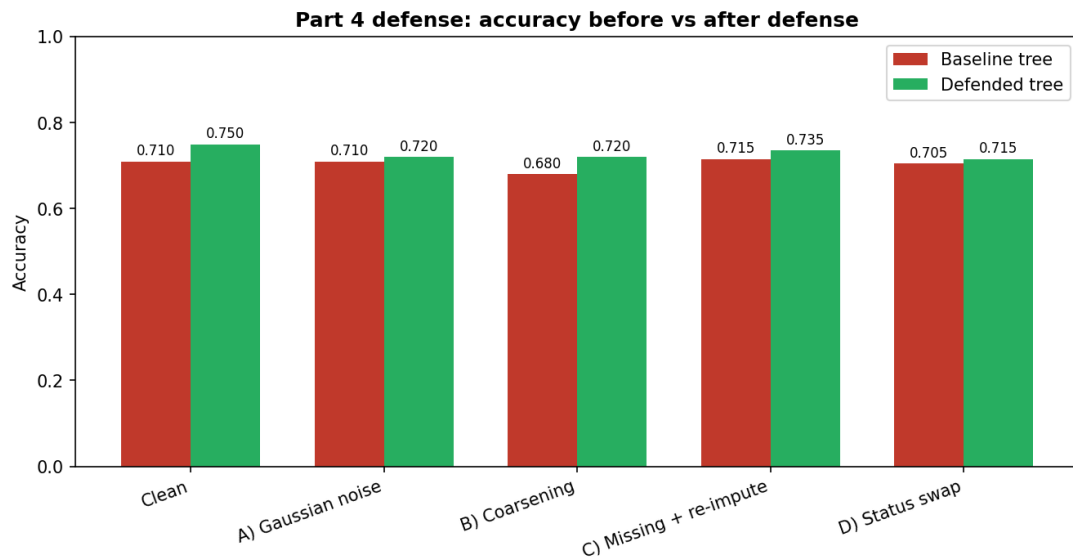


Figure 10: Accuracy before and after the defense. The defended tree improves accuracy in all five scenarios. The biggest gain is under coarsening, where accuracy rises from 0.68 to 0.72.

The defense improves the clean test accuracy from 0.71 to 0.75. It also improves the average accuracy across the four perturbed test sets from 0.7025 to 0.7225. The worst stressed accuracy also improves, from 0.68 in the baseline to 0.715 after the defense.

Macro F1 improves in every scenario as well. This matters because macro F1 gives equal weight to the Bad and Good classes, so it is more honest than accuracy when the classes are imbalanced.

There is one mixed result in the flip rates. Under coarsening, the flip rate drops strongly from 15% to 7%, which is a good sign of better stability. But under the status-swap scenario, the defended model has a higher flip rate (15.5%) than the baseline (9.5%). So the defense improves performance overall, but it does not remove all instability.

Analysis

Question 1: Did the defense improve robustness?

Yes, overall it did.

The clearest evidence is the perturbed-data performance. The average accuracy across the four stressed test sets rises from 0.7025 to 0.7225. The worst-case stressed accuracy improves from 0.6800 to 0.7150. The largest gain appears in Scenario B (coarsening), where the model goes from 0.68 to 0.72 accuracy and the flip rate falls from 15% to 7%.

This means the defended tree handles noisy or degraded inputs better than the original tree. It is less dependent on the exact clean version of the features because it has already seen similar distortions during training.

Question 2: Did performance decrease?

Not in the main headline metrics. Clean accuracy increases from 0.71 to 0.75, and clean macro F1 increases from 0.5381 to 0.6573. So overall performance improves rather than decreases.

However, there is still a trade-off inside the class-level behaviour. On clean data, recall for the Good class falls slightly from 0.9429 to 0.9071. At the same time, recall for the Bad class rises much more strongly from 0.1667 to 0.3833. In simple words, the defended model becomes less likely to approve almost everyone, and better at catching risky applicants.

So the answer is: no major overall decrease, but yes, one part of performance (Good-class recall) becomes slightly lower in exchange for much better balance.

Question 3: Why is this trade-off expected?

This trade-off is expected because a defense usually makes a model less tuned to perfect clean data and more prepared for messy real data.

When we train on noisy, rounded, incomplete, and manipulated examples, the tree learns broader and safer decision rules. That helps robustness, but it also makes the model more conservative. Instead of approving borderline applicants as easily as before, it becomes more careful. This can reduce recall for the majority Good class, even while improving robustness and Bad-class detection.

In a lending setting, this trade-off is reasonable. Missing risky applicants is costly, so a model that is slightly less generous but more stable and better at finding bad-credit cases is safer for deployment than a fragile model that only looks good on clean data.

Part 5: Analysis and Recommendation (25%)

Question 1: Is your model robust enough for real-world use?

No. The baseline correctly identifies only 17% of risky applicants, and Scenario B (coarsening) widens the demographic parity gap by sex from 0.086 to 0.476 while flipping 15%

of all predictions. The defended model improves on these numbers, clean accuracy rises to 0.75 and worst-case stressed accuracy to 0.715; but it does not eliminate the risks described below, and neither version meets the bar for reliable real-world deployment.

Question 2: Which risks remain?

Six material risks persist even after the defense. Bad-class recall reaches only 0.38, meaning over 60% of genuinely risky applicants are still approved. Fairness under coarsening was not re-evaluated on the defended model, so whether the DP gap of 0.476 improves is unknown. The flip rate under status swap rises from 9.5% to 15.5% after the defense, making the model more vulnerable to deliberate feature manipulation. Female applicants carry a Brier score 35.8% higher than males even on clean data, meaning probability outputs are systematically less reliable for them. The dataset dates from the 1990s and may not reflect modern credit behaviour. Finally, the Decision Tree produces no uncertainty signal. Every borderline case receives the same hard approve or reject as a clear-cut one.

Question 3: Did your defense meaningfully help?

Yes, in a limited but genuine way. Accuracy improves across all five test scenarios (+0.01 to +0.04), macro F1 improves across all five (+0.051 to +0.119), the coarsening flip rate halves from 15% to 7%, and Bad-class recall more than doubles from 0.17 to 0.38. These are real gains, not cosmetic ones. However, the defense does not fix calibration for female applicants, does not close the Bad-class recall gap below 60%, and makes the model more sensitive to categorical manipulation in Scenario D. It is a meaningful first step that would need to be paired with the monitoring and validation measures in Question 5 before deployment.

Question 4: Which applications require stronger robustness?

Robustness requirements vary by deployment context. Fully automated loan approval or fraud flagging with no human review is the most demanding setting: a single corrupted input can cause an irreversible outcome, and the model studied here does not meet this bar. Decision-support tools where a loan officer reviews the model's output before acting have more tolerance for imprecision, since the reviewer catches boundary errors; the defended model is closer to meeting this bar if borderline cases are routed to human review. Batch portfolio analytics, where errors in individual predictions partially cancel at the aggregate level, is the least demanding context and may be adequate with appropriate statistical caveats. Credit scoring typically falls in the first category, which requires Bad-class recall above 0.60, certified fairness under realistic data quality variation, and a formal model card before any deployment decision.

Question 5: Would you deploy this model? Under what conditions?

We would not deploy this model in a fully automated pipeline. The defended model could serve as a risk-flagging tool in a human-in-the-loop process, subject to five conditions. First, any applicant whose predicted probability falls between 0.40 and 0.60 must receive human review before a decision is issued. Second, numeric features, especially `duration` and `amount` must be validated against plausible ranges before scoring, with implausible values flagged rather than scored silently. Third, selection rates and equalized odds gaps must be monitored continuously by sex, age group, and foreign worker status, with automated alerts if any demographic parity gap exceeds 0.10. Fourth, the model must be recalibrated on recent data every six months, and any production deployment must first retrain on a contemporary dataset rather than the 1990s German Credit data. Fifth, applicants who receive a rejection must have access to a decision explanation drawn from the actual tree path for their input, not from global Gini importance scores, and a formal appeal process. Under these conditions the model is usable as a decision-support filter; it should not operate as a standalone decision-maker.